Studienarbeit

Wissenschaftliche Arbeit Template

des Studiengangs Informatik

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Luis Bros

Juni 2025

Matrikelnummer	123456
Kurs	TINF22C
Betreuer	Prof. Max Mustermann

Ehrenwörtliche Erklärung

Ich versichere hiermit ehrenwörtlich, dass ich meine wissenschaftliche Arbeit mit dem Thema „…“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Diese Arbeit wurde bisher in gleicher oder ähnlicher Form oder auszugsweise noch keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Ort, Datum

Unterschrift

Abstract

Diese Arbeit erforscht wie man tolle LaTeX-Templates für wissenschaftliche Arbeiten erstellen kann.

Abbildungsverzeichnis

1. Allgemeines Datenflussdiagramm von den Eingabedaten der Endnutzenden zu den Ausgabedaten des Betreibers bei LLM-Servicemodellen. 5
2. Die Abfolge der fünf Projektphasen des Wasserfallmodells. 9

Tabellenverzeichnis

Abkürzungsverzeichnis

LLM Large Language Model

NLP Natural Language Processing

FF Forschungsfrage

FOSS Free and Open-Source Software

Inhaltsverzeichnis

1. Einleitung **1**
1.1. Problemstellung 1
1.2. Forschungsfragen 1

2. Grundlagen **2**
2.1. Genutzte Technologien 2
2.2. LLMs in der Umfrageerstellung 2
2.2.1. Hardware Anforderungen 3
2.2.2. Datenschutz 3
2.2.3. Prompt Engineering 6

3. Anforderungen **9**

4. Umsetzung **10**

5. Evaluation **11**

6. Zusammenfassung, Reflexion & Ausblick **12**

A. Anhang **14**

1. Einleitung

Hier beginnt die Einleitung der Studienarbeit. Sie gibt einen Überblick über das Thema, die Problemstellung und die Forschungsfrage. Die Einleitung sollte den Leser in das Thema einführen und die Relevanz der Arbeit verdeutlichen.

1.1. Problemstellung

Das Problem, wieso diese Arbeit überhaupt geschrieben wird. Ein Zitat ist hier passend, um die Relevanz des Themas zu verdeutlichen. Es sollte klar werden, warum das Thema wichtig ist und welche Lücken in der bestehenden Forschung bestehen.

1.2. Forschungsfragen

Diese Arbeit setzt sich als Ziel, die folgenden Forschungsfragen (FF) zu beantworten:

FF1: Welche Kriterien sind relevant für ein Template?

Diese Forschungsfrage gilt es zu beantworten. Dafür muss mit der wissenschaftlichen Methode der Literraturrecherche, folgendes erarbeitet werden:

2. Grundlagen

In diesem Kapitel werden die grundlegenden Technologien und Methoden erläutert, auf denen die vorliegende Seminararbeit basiert.

2.1. Genutzte Technologien

Verwendete Technologien...

2.2. LLMs in der Umfrageerstellung

Large Language Models (LLMs) *(dt. große Sprachmodelle)* sind KI-Modelle, die auf großen Textmengen trainiert wurden und in der Lage sind, menschenähnliche Texte zu generieren. In den letzten Jahren wurden große Fortschritte in dem Feld des Natural Language Processing (NLP) erzielt, was zu einem großen Anstieg der Nutzung von KI-Chatbots geführt hat. Insbesondere mit der Veröffentlichung von OpenAIs ChatGPT im November 2022 wurde ein neuer Meilenstein erreicht. LLMs haben sich als äußerst leistungsfähig erwiesen und sind in der Lage, komplexe Aufgaben zu bewältigen, die zuvor menschliche Intelligenz erforderten. Diese Modelle können nicht nur Texte generieren, sondern auch Fragen beantworten, Texte zusammenfassen und sogar kreative Inhalte erstellen. [1]

Die Entwicklung von LLMs kann in drei Phasen unterteilt werden [1]:

1. **Training Phase:** In dieser Phase werden die Modelle auf großen vorbereiteten Textmengen trainiert, um die Muster und Strukturen der Sprache zu lernen. Das Training erfolgt in einem unüberwachtem Lernprozess basierend auf der Transformer Architektur, die 2017 von Vaswani et al. eingeführt wurde. Diese Architektur stützt sich auf einen „Attention Mechanism“, um den Kontext von Wörtern in einem Satz zu verstehen und relevante Informationen zu extrahieren. [2] Dabei werden die erlernten Informationen numerisch in den Parametern des Modells gespeichert. Die Größe der Modelle wird in der Regel in Milliarden von Parametern angegeben, was auf die Komplexität und den Umfang des Modells hinweist. [1]

2. **Fine-Tuning:** In dieser Phase wird das vortrainierte Modell auf spezifische Aufgaben oder Domänen angepasst. Dies geschieht häufig durch überwachtes Lernen, bei dem das Modell mit gekennzeichneten Daten trainiert wird. Weitere bekannte Methoden sind Reinforcement Learning from Human Feedback (RLHF), Trans-

fer Learning und Feedback Loops, wo durch Nutzereingaben und Rückmeldungen das Modell kontinuierlich verbessert wird. [3]

3. **Inferenz:** In dieser Phase wird das trainierte Modell verwendet, um Vorhersagen oder Texte zu generieren. Dies geschieht in der Regel in Echtzeit und erfordert eine effiziente Hardware-Infrastruktur.

Da LLMs in der Lage sind, menschenähnliche Sprache zu generieren und komplexe Zusammenhänge zu verstehen, eignen sie sich hervorragend für die Unterstützung bei der Umfrageerstellung. Mit dem entsprechenden Hintergrundwissen über die Zielgruppe sind sie in der Lage, präzise und relevante Fragen für die Nutzenden zu formulieren. Mit dieser Funktion können sie aus einem trivialem Prompt den Kontext einer Umfrage analysieren und passende Fragen erstellen. Durch die Automatisierung wird der Prozess der Umfrageerstellung erheblich beschleunigt und vereinfacht. Neben dem genannten Vorteil besteht zudem die Möglichkeit, dass LLMs qualitativ hochwertigere Fragen generieren können. Durch ihr Vorwissen über die objektiven Richtlinien zur Umfragenerstellung sind sie in der Theorie in der Lage, präzisere und weniger verwirrende Fragen mit passenden Antworttypen zu formulieren.

2.2.1. Hardware Anforderungen

Der größte Aufwand an Energie, Zeit und Rechenleistung wird in der Trainingsphase benötigt. Die Inferenzphase ist im Vergleich dazu deutlich weniger ressourcenintensiv. Die benötigte Rechenleistung hängt von der Größe des Modells ab. Kleinere Modelle benötigen weniger Rechenleistung, während größere Modelle mehr Ressourcen erfordern. Entscheidend, ob ein Modell überhaupt ausgeführt werden kann, ist die Größe des RAMs, weil beim Start das ganze Modell in den RAM geladen wird. Bei Maschinen mit GPU wird meist der VRAM benutzt. Wenn keine GPU vorhanden ist, muss auf die CPU und den DRAM zurückgegriffen werden. Schon bei einer Modellgröße von sieben Milliarden Parametern werden über 14GB RAM benötigt. Diese Anforderungen übersteigen meist die Kapazität von mobilen Endgeräten. [4]

Eine weitere Limitierung ist der benötigte Speicherplatz und die Netzwerkbandbreite. Die Modelle sind in der Regel mehrere Gigabyte groß und benötigen eine schnelle Internetverbindung, um effizient heruntergeladen zu werden. [5]

2.2.2. Datenschutz

Der EU AI Act ist seit 2024 eine Verordnung der Europäischen Union, die darauf abzielt, einen einheitlichen Rechtsrahmen für Künstliche Intelligenz zu schaffen. Er legt

Anforderungen und Standards für KI-Systeme fest, um sicherzustellen, dass sie sicher, transparent und ethisch sind. Der EU AI Act unterscheidet zwischen verschiedenen Risikoklassen von KI-Systemen und legt spezifische Anforderungen für jedes Risiko fest. Alle KI-Systeme, die in der EU angeboten und betrieben werden, müssen sich an diese Verordnung halten. [3]

Nach Artikel 3 Absatz 3 des EU AI Act wird ein Anbieter als die juristische Person, die ein KI-System entwickelt und bereitstellt. Nach Artikel 3 Absatz 4 ist ein Betreiber die juristische Person, die ein KI-System für Endnutzende implementiert und bereitstellt. [6]

LLMs können in der Regel auf drei verschiedene Wege aufgerufen werden [3]:

1. **LLM as a Service:** In diesem Servicemodell gibt der Bereitstellende den Zugriff auf sein LLM über eine API frei. Dabei läuft das Modell auf fremder Infrastruktur. Die Nutzenden können Anfragen an das Modell über die bereitgestellte API senden und erhalten die generierte Antwort zurück. Diese Methode ist simpel zu implementieren, da die Entwicklenden keine eigene Infrastruktur bereitstellen müssen. Oft wird für die Anzahl der Eingabe- und Ausgabetokens eine „Pay as you go“-Gebühr erhoben. Bei dieser Art von Service gibt es wiederum drei verschiedene Arten von Modellen, aus welchen die Entwickelnden auswählen können:

    - **Proprietäres LLM:** Die Entwickelnden haben keine Einsicht auf die Modellarchitektur und können diese auch nicht beeinflussen. Ein bekanntes Beispiel ist OpenAIs GPT-API.

    - **Fine-tuneable LLM:** Die Entwickelnden haben die Möglichkeit, das Modell mit eigenen Daten zu fine-tunen. Dies geschieht in der Regel über eine API, die es den Entwickelnden ermöglicht, ihre Daten hochzuladen und das Modell anzupassen. Allerdings können die Entwickelnden die Modellarchitektur wieder nicht einsehen. Ein Beispiel dafür ist der Azure OpenAI Service.

    - **Open-Source LLM:** Die Entwickelnden können das Modell frei von einem beliebigen Anbieter wählen und haben die Möglichkeit, das Modell nach Belieben anzupassen und einzusehen. Unter anderem bietet HugginFace mit der serverless Inference-API die Möglichkeit, Open-Source Modelle zu hosten und diese über eine API anzusprechen. Nur hier sind Anbieter und Betreiber nicht identisch.

2. **Self-deployed LLM:** Hier können die Entwickelnden ein beliebiges Open-

Source Modell oder sogar ein selbst trainiertes Modell auf ihrer eigenen Infrastruktur hosten. So werden die Entwickelnden zum Betreiber und eventuell zum Anbieter gleichzeitig. Dies erfordert jedoch mehr technisches Wissen und Ressourcen, da die Entwickelnden für die Bereitstellung und Wartung des Modells verantwortlich sind. Diese Methode bietet jedoch volle Kontrolle über die Daten, die Modellarchitektur und den Datenfluss. Keine Nutzerdaten werden an Dritte weitergegeben.

3. **On-Device LLM:** Hierbei sind die Endnutzenden die Betreiber des Modells. Das Modell wird lokal auf dem Endgerät der Nutzenden ausgeführt, was bedeutet, dass keine Daten an Dritte oder sogar Zweite weitergegeben werden. Bei dieser Methode haben theoretisch die Endnutzenden volle Einsicht auf die Modellarchitektur.

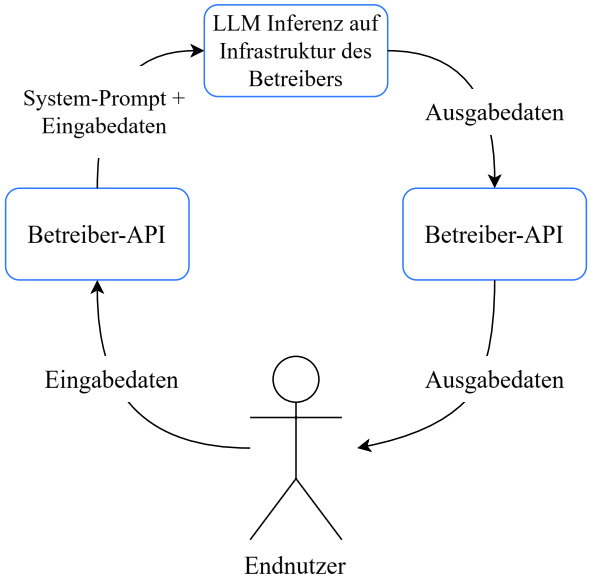

Abbildung 1: Allgemeines Datenflussdiagramm von den Eingabedaten der Endnutzenden zu den Ausgabedaten des Betreibers bei LLM-Servicemodellen. Eigene Abbildung basierend auf [3].

Der Datenfluss zwischen den Endnutzenden und dem Betreiber in allen LLM-Servicemodellen läuft wie in Abbildung 1 dargestellt ab. Je nach Servicemodell unterscheidet sich der Betreiber. Erst schicken die Endnutzenden ihre Anfrage mit den Eingabedaten an die API des Betreibers. Der Betreiber kann entweder ein externer Dienstleister, der Betreiber der Webseite oder die Endnutzenden selbst sein. Unter die Eingabedaten fallen der User-Prompt, Audioaufnahmen, Bilder sowie alle sonstigen Arten von Dateien, die die Nutzenden hochladen. Die API des Betreibers empfängt die Anfrage und übergibt diese in Kombination mit dem System-Prompt an das LLM. Das Modell generiert eine Antwort und sendet diese zurück an die Betreiber-API. Letztendlich wird von dort die Antwort an die Endnutzenden weitergeleitet. Vor und

nach der Inferenzphase ist es möglich, dass der Betreiber die Ein- und Ausgabedaten und weitere personenbezogene Metadaten speichert. Diese Daten können dann für verschiedene Zwecke verwendet werden, wie zum Beispiel zur Verbesserung des Modells oder zur Analyse von Nutzerverhalten. In der Theorie können die Daten auch zur Monetarisierung weiterverarbeitet werden. [3]

In Zuge dessen entstehen für die Endnutzenden mehrere Risiken, unter anderem [3]:

- **Offenlegung sensibler Daten:** Nutzende können versehentlich oder aus Vertrauen sensible personenbezogene Daten preisgeben.
- **Unbefugter Zugriff:** Unzureichend gesicherte Schnittstellen können Unbefugten Zugriff auf Nutzerkonten geben und somit letzte Anfragen einsehbar machen.
- **Datenmissbrauch:** Oftmals ist es unklar, wie die Daten verarbeitet und gespeichert werden. Die Nutzenden können nie sicher sein, was mit ihren Daten passiert. Personenbezogene Daten können sich ohne Zustimmung in einem Trainingsdatensatz wiederfinden oder an Dritte weitergegeben werden.
- **Datenlecks:** Wenn der Betreiber die Daten nicht ausreichend schützt, können diese in die Hände Dritter gelangen. Dies kann zu Identitätsdiebstahl oder anderen Formen des Missbrauchs führen.

Obwohl es für den Schutz dieser Daten klare gesetzliche Vorgaben nach dem EU AI Act und der Datenschutz-Grundverordnung (DSGVO) gibt, ist es für die Nutzenden oft schwierig, diese zu verstehen und zu überprüfen. Auch ist es durch die globale Gegebenheit des Internets oft schwer herauszufinden, welchen nationalen Regelungen der Betreiber unterliegt. Diese Risiken können alleinig durch On-Device LLMs mitigiert werden. [3]

2.2.3. Prompt Engineering

Prompt Engineering ist eine entscheidende Methodik in der Arbeit mit LLMs, die jede Person bewusst oder unterbewusst bei jeder Anfrage verwendet. Ein Prompt (*dt. Eingabeaufforderung*) ist eine aufgabenspezifische Textanweisung, die dem Modell zur Inferenz übergeben wird, um die Ausgabe zu steuern, ohne die Modellparameter zu verändern. Im Kern des Prompt Engineering steht das Bestreben, durch methodisch geformte Texteingaben die Ausgaben künstlicher Intelligenz zu optimieren. [7]

Zu den bekanntesten Techniken des Prompt Engineering gehören [7]:

- **Zero-Shot Prompting:** Bei dieser Technik wird das Modell mit einer Aufgabe konfrontiert, ohne dass es vorherige Beispiele oder Anweisungen erhält. Es wird

erwartet, dass das Modell die Aufgabe aufgrund des vorhandenen Wissens und des gegebenen Textes versteht und ausführt. Beispiel Prompt: „`Schreibe einen Text über die Vor- und Nachteile von LLMs.`“

- **Few-Shot Prompting:** Hierbei werden dem Modell einige Beispiele der Aufgabe gegeben, um ihm zu helfen, die Struktur und den Kontext der gewünschten Ausgabe zu verstehen. Diese Technik ist besonders nützlich, wenn das Modell Schwierigkeiten hat, die Aufgabe allein zu bewältigen oder aus vorgegebenen Auswahlmöglichkeiten auswählen muss. Allerdings können durch die gegebenen Beispiele häufige Wörter bevorzugt werden oder die Kreativität sinken. Beispiel Prompt: „`Beispiel 1: Frage: Was ist die Hauptstadt von Frankreich? Antwort: Paris. Beispiel 2: Frage: Was ist die Hauptstadt von Italien? Antwort: Rom. Aufgabe: Was ist die Hauptstadt von Spanien? Antwort:`“

- **Chain-of-Thought Prompting:** Diese Technik ermutigt das Modell, seine Schlussfolgerungen und Überlegungen während des Problemlösungsprozesses zu erläutern. Diese Methode generiert eine Kette von Gedanken, also Argumentationsschritte, um so Probleme schrittweise zu lösen. Beispiel Prompt: „`Schritt 1: Erkläre was LLMs sind. Schritt 2: Erkläre welche Einsatzgebiete sie haben. Schritt 3: Erkläre wie sie funktionieren. Basierend auf den Schritten, erkläre was die Vor- und Nachteile sind.`“

- **Role Prompting:** Bei dieser Technik wird das Modell in eine bestimmte Rolle versetzt, um die Ausgabe zu steuern. Dies kann durch die Verwendung von Expertenrollen geschehen, die das Modell dazu bringen, aus der Perspektive einer bestimmten Person oder Entität zu antworten. [8] Beispiel Prompt: „`Du bist ein guter Grundschullehrer und kannst Probleme einfach runterbrechen. Erkläre einem Grundschüler, was LLMs sind und wie sie funktionieren.`“

- **Self-Consistency Prompting:** Diese Technik ermutigt das Modell, mehrere Antworten auf eine Frage zu generieren und dann die beste Antwort auszuwählen. Für komplexe Aufgaben mit mehreren validen Lösungswegen, werden diese verglichen und die schlüssigste Antwort gewählt. Beispiel Prompt: „`Erkläre was LLMs sind. Generiere 5 verschiedene Erklärungen und wähle die schlüssigste und klarste aus.`“

In dem Prompting Framework „TACOMORE“ (Task, Context, Model, Reproducibility) gehört neben dem Definieren von einer oder mehreren Aufgaben mittels oben genannten Techniken auch das Bereitstellen von passenden Kontextinformationen, die

Auswahl des richtigen Modells und die Definition des gewünschten Ausgabeformats, um Reproduzierbarkeit sicherzustellen. Wegen der Natur von LLMs kann nicht bei jeder Ausführung desselben Prompts dieselbe Ausgabe garantiert werden. Deswegen ist es wichtig, dass man das Ausgabeformat festlegt. Maschinenlesbare Formate wie JSON oder XML sind passend, da der Datentyp in einem Schlüssel-Wert-Verfahren festgelegt werden kann. [9]

Durch die Kombination der oben genannten Techniken und Frameworks kann die Leistung und die Genauigkeit erheblich verbessert werden. Es ist wichtig, dass die Prompt Engineering Techniken an die spezifischen Anforderungen und Ziele der Anwendung angepasst werden, um die besten Ergebnisse zu erzielen. Besonders bei Anwendungsfällen, wo die Nutzenden die Ausgabe nicht direkt sehen und keine Möglichkeit haben, weitere Prompts nachzusenden, um die Ausgabe zu verbessern, ist Prompt Engineering entscheidend. Das Ausgabeformat und die Qualität müssen auf den ersten Versuch stimmen. Sonst kann es zu Unzufriedenheit oder sogar Fehlern kommen. So ein Anwendungsfall ist unter anderem die Integration von KI in ein Umfrage-Tool.

3. Anforderungen

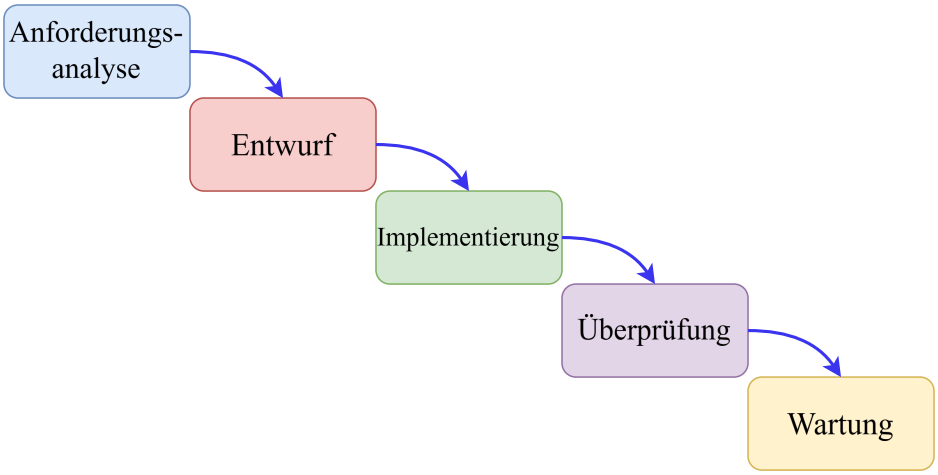

Abbildung 2: Die Abfolge der fünf Projektphasen des Wasserfallmodells. Eigene Abbildung basierend auf [10].

Was sind die Anforderungen an dein Projekt? Mit welchem Vorgehensmodell wurde es Umgesetzt?

4. Umsetzung

Die nachfolgenden Abschnitte widmen sich der detaillierten Darstellung der praktischen Umsetzung der entwickelten Software. Zunächst wird der Entwurf der Softwarearchitektur beschrieben, welcher die zweite Phase des Wasserfallmodells repräsentiert. Im Anschluss daran erfolgt die Erläuterung der Implementierung im Rahmen der dritten Phase des Wasserfallmodells. Im Zuge der Implementierung werden zudem die aufgetretenen Herausforderungen sowie die jeweils erarbeiteten Lösungsansätze systematisch aufgezeigt und erläutert.

5. Evaluation

Wissenschaftliche Methodik wird angewendet, um die Forschungsfragen umfassend zu beantworten. In den vorangegangenen sowie den nachfolgenden Kapiteln werden die vier Forschungsfragen systematisch untersucht:

Zunächst wurde die erste Forschungsfrage durch eine ausführliche Literaturrecherche evaluiert. Die dabei gewonnenen Erkenntnisse und Ergebnisse sind im Grundlagenkapitel 2.1 detailliert zusammengetragen und dokumentiert worden.

6. Zusammenfassung, Reflexion & Ausblick

Abrundender Satz für die Zusammenfassung, der die Relevanz des Themas und die Bedeutung der Ergebnisse unterstreicht.

Zusammenfassende Erkenntnisse der Forschungsfragen:

Reflexion: Was sollten Peers beachten, die das gleiche vorhaben? Was waren die Learnings/Challenges

Ausblick: Was könnte weiter erforscht werden mit unbegrenzten Ressourcen?

Literatur

[1] Y. Liu *et al.*, "Understanding LLMs: A Comprehensive Overview from Training to Inference," Jan. 2024. [Online]. Available: https://arxiv.org/abs/2401.02038

[2] A. Vaswani *et al.*, "Attention Is All You Need," Aug. 2023. [Online]. Available: https://arxiv.org/abs/1706.03762

[3] I. Barbera, *AI Privacy Risks and Mitigations - Large Language Models (LLMs).* European Data Protection Board, Apr. 2025. [Online]. Available: https://www.edpb.europa.eu/system/files/2025-04/ai-privacy-risks-and-mitigations-in-llms.pdf

[4] K. Alizadeh *et al.*, "LLM in a flash: Efficient Large Language Model Inference with Limited Memory," 2024. [Online]. Available: https://arxiv.org/abs/2312.11514

[5] C. F. Ruan *et al.*, "WebLLM: a High-Performance In-Browser LLM inference Engine," Dec. 2024. [Online]. Available: https://arxiv.org/abs/2412.15803

[6] European Parliament and the Council of the European Union, "Artificial Intelligence Act," Jun. 2024. [Online]. Available: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32024R1689

[7] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications," Feb. 2024. [Online]. Available: https://arxiv.org/abs/2402.07927

[8] A. Kong *et al.*, "Better Zero-Shot Reasoning with Role-Play Prompting," Aug. 2023. [Online]. Available: https://arxiv.org/abs/2308.07702

[9] B. Li and H. Wang, "TACOMORE: Leveraging the Potential of LLMs in Corpus-based Discourse Analysis with Prompt Engineering," Dec. 2024. [Online]. Available: https://arxiv.org/abs/2412.10139

[10] K. Pfetzing and A. Rohde, *Ganzheitliches Projektmanagement*, 3rd ed. Versus, 2009, ISBN: 9783039091430.

A. Anhang

Java Code

```
public static void main(String[] args) {
    System.out.println("Hello, World!");
}
```

Quelltext 1: Java Hello World.